phone_app/lib/phone_app/conversations/query/sms_message_store.ex
```
def get_sms_message!(id) do
  Repo.get!(SmsMessage, id)
end
```

We're almost done. Next, we'll integrate this job into the SMS creation flow.

## Enqueue a Job After SMS Creation

Our job should be enqueued when an SMS message is created with certain properties (status = queued and direction = outgoing). We already have a create_sms_message/1 function that all SMS messages are created through, but it's a defdelegate. If we want to hook into it, we need to turn the defdelegate into a regular function.

In the lib/phone_app/conversations/conversations.ex file, delete the existing create_sms_message delegate and add the following function:

phone_app/lib/phone_app/conversations/conversations.ex
```
def create_sms_message(params) do
  PhoneApp.Repo.transaction(fn ->
    with {:ok, message} <- Query.SmsMessageStore.create_sms_message(params),
         {:ok, _} <- maybe_enqueue_status_worker(message) do
      message
    else
      {:error, cs} -> PhoneApp.Repo.rollback(cs)
    end
  end)
end

defp maybe_enqueue_status_worker(message) do
  case message do
    %{direction: :outgoing, status: "queued"} ->
      PhoneApp.Conversations.Worker.StatusWorker.enqueue(message)

    _ ->
      {:ok, :skipped}
  end
end
```

This function looks a little dense at first, so let's break it apart.

We start by wrapping the entire function body inside of Repo.transaction/1. This function wraps the provided function inside of a SQL transaction. If an error is raised or Repo.rollback/1 is called, the transaction rolls back and returns {:error, tuple}. If the function doesn't error, then the return value is wrapped in an {:ok, tuple}. This transaction is optional—you could insert the Oban job without one. However, it's usually important to guarantee that jobs are created alongside data.